

04

JavaScript Biblioteki. API.

Aplikacje Internetowe 1
dr inż. Artur Karczmarczyk



<https://ispot.link/lista>

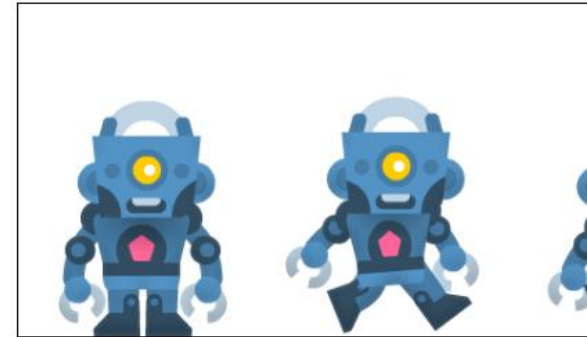


<https://ispot.link/ai1pre5>

Agenda

- Zdarzenia i ich obsługa
- Połączenia asynchroniczne: XMLHttpRequest i Fetch API
- Grafika rastrowa w Canvas
- Grafika wektorowa w SVG
- Animacje SVG
- Drag-and-Drop API
- Obsługa multimedialnych
- Geolokalizacja i mapy

Size 1:1



Scale 1:4

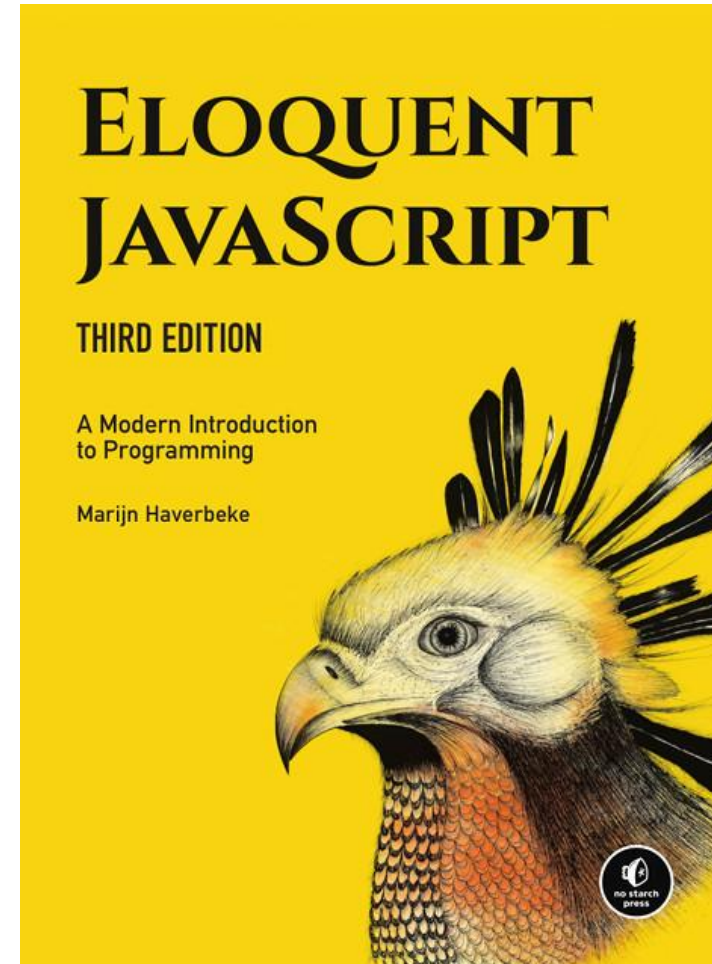


Single Image



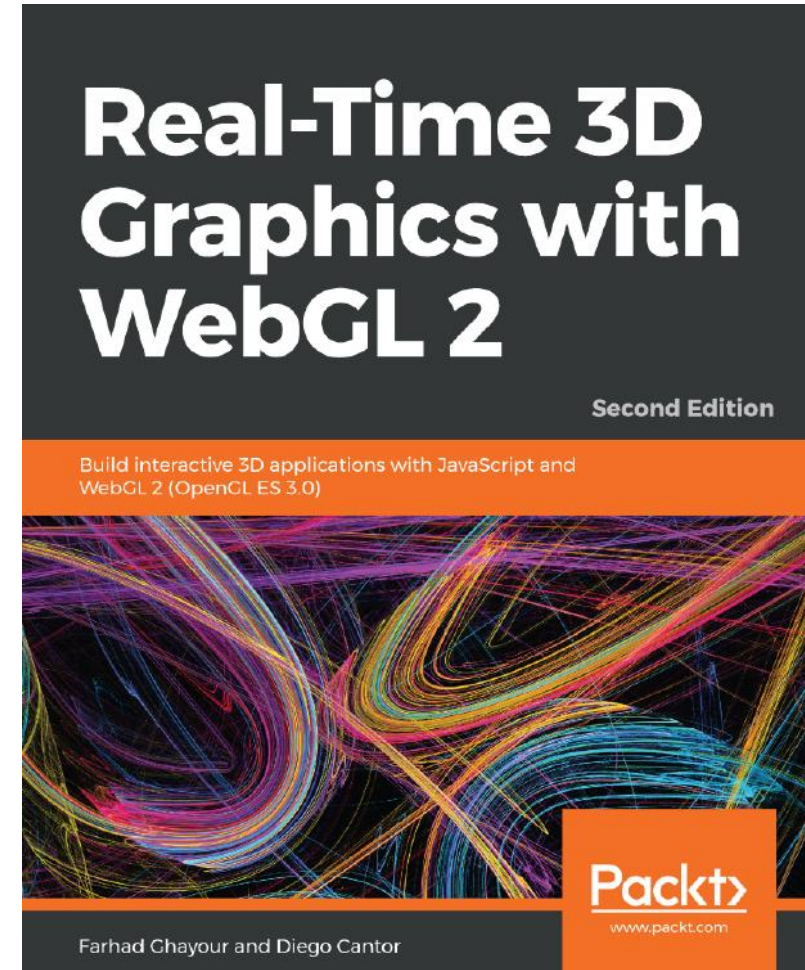
Literatura

- Eloquent JavaScript
 - Third Edition
 - Marjin Haverbeke
 - <http://eloquentjavascript.net/>
 - licencje:
 - <http://creativecommons.org/licenses/by-nc/3.0/>
 - <http://opensource.org/licenses/MIT>



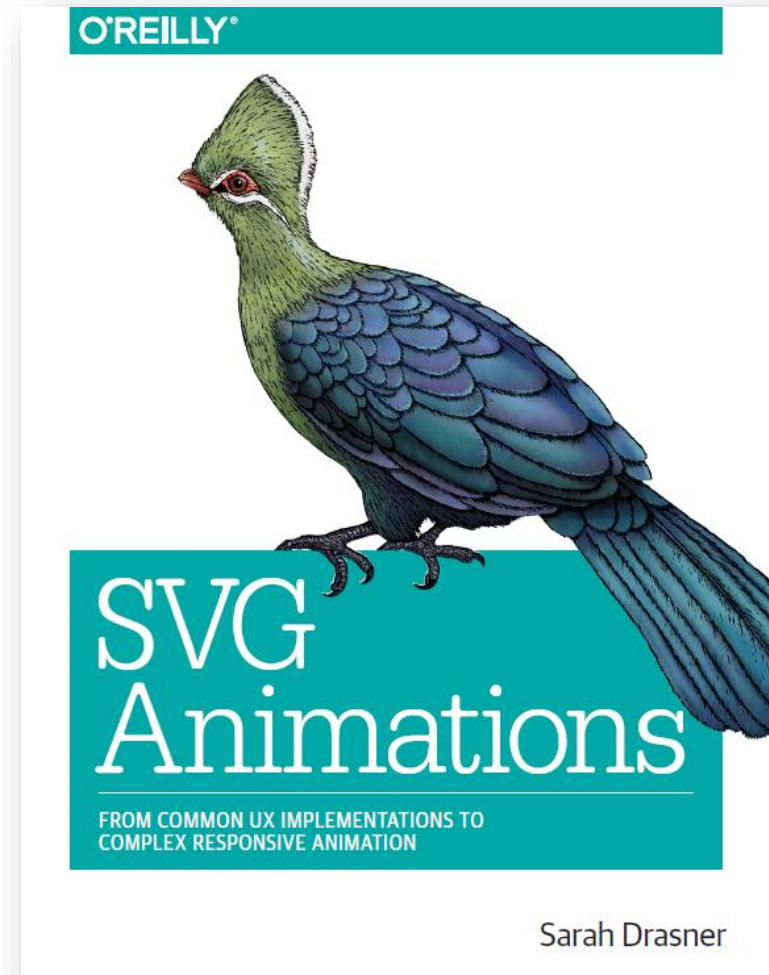
Literatura

- Real-Time 3D Graphics with WebGL 2
 - Second Edition
 - Farhad Ghayour
 - Diego Cantor
 - 2018 Packt Publishing



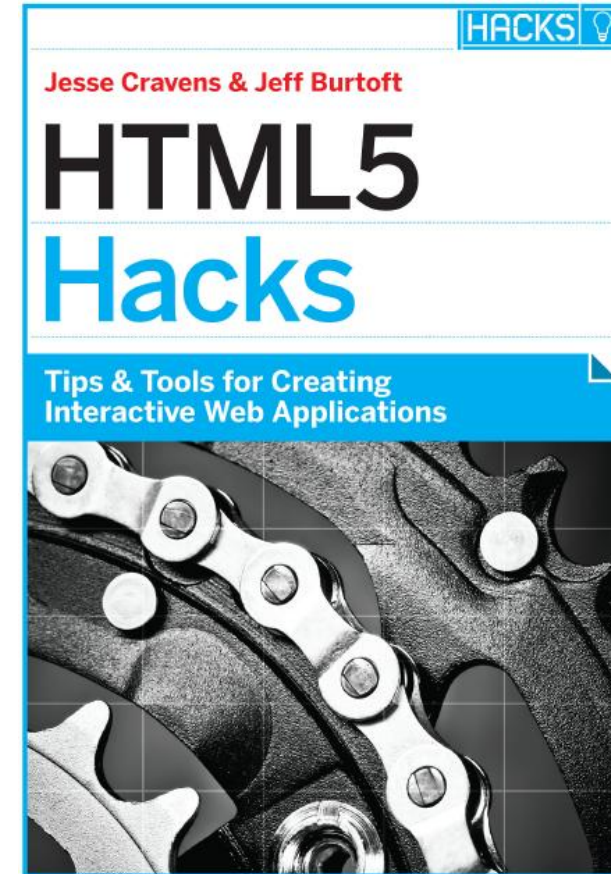
Literatura

- SVG Animations
 - From Common UX Implementations to Complex Responsive Animation
 - Sarah Drasner
 - 2017 O'Reilly Media, Inc.



Literatura

- HTML5 Hacks
 - Tips & Tools for Creating Interactive Web Applications
 - Jesse Cravens & Jeff Burtoft
 - 2013 O'Reilly Media, Inc.



O'REILLY®

24.10.2025

Obsługa zdarzeń

Wyobraźmy sobie...

- Aplikacja, która powinna reagować na naciśnięcie klawisza.
- Jak sprawdzić, czy klawisz naciśnięty?

- Alice:

- można ustawić setpoint i sprawdzać co kilka milisekund stan wciśnięcia klawisza
- co jeśli wykonuje się zbyt późno?
- co jeśli do sprawdzenia klawiszy?
- słabo...



- Bob:

- mógłby istnieć interfejs sprzętowy
- naciśnięcie klawisza dodaje klawisz do kolejki
- program co chwilę sprawdza kolejce
- tzw. „polling”
- lepiej, ale wciąż nie to...



Zdarzenia

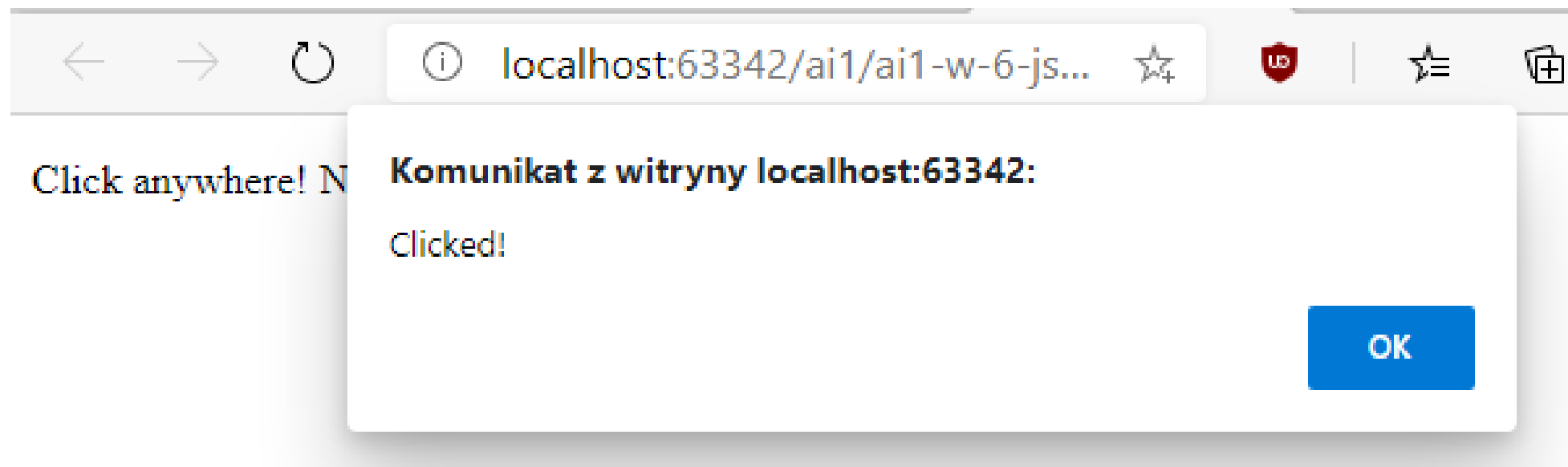
- Lepiej, żeby system aktywnie powiadamiał kod o wystąpieniu zdarzenia.
- Przeglądarki pozwalają zarejestrować funkcje obsługujące określone zdarzenia.
- Wzorzec obserwator.
- Obiekt `window` reprezentuje okno przeglądarki z dokumentem.

```
<p>Click anywhere! Not necessarily on text!</p>  
<script>  
  window.addEventListener('click', () => {  
    alert('Clicked!');  
  })  
</script>
```

```
<p>Click anywhere! Not necessarily on text!</p>  
<script>  
  window.addEventListener('click', function() {  
    alert('Clicked!');  
  })  
</script>
```

```
<p>Click anywhere! Not necessarily on text!</p>  
<script>  
  let onClickCallback = function() {  
    alert('Clicked!');  
  }  
  window.addEventListener('click', onClickCallback); //no brackets()  
</script>
```

Zdarzenia



Zdarzenia a elementy drzewa DOM

- `addEventListener` na `window` odnosi się do całego okna
- `addEventListener` można wykonać na poszczególnych elementach drzewa DOM
- Funkcje obserwujące zdarzenie wykonają się tylko jeśli zdarzenie zajdzie w kontekście obiektu, na którym obserwator jest zarejestrowany.

```
<p id="first">Click here for event 1</p>
<p id="second">Click here for event 2</p>
<p id="third">Click here but nothing will happen!</p>
<script>
  let p1 = document.getElementById("first");
  let p2 = document.getElementById("second");

  p1.addEventListener('click', function() {
    alert('Clicked 1!');
  })
  p2.addEventListener('click', function() {
    alert('Clicked 2!');
  })
</script>
```

Kilka obserwatorów na jednym elemencie

```
<p>Click here and see what happens!</p>  
<script>  
  let paragraph = document.querySelector("p");  
  
  paragraph.addEventListener('click', function() {  
    alert('First listener reacted!');  
  })  
  paragraph.addEventListener('click', function() {  
    alert('Second listener reacted!');  
  })  
</script>
```


onclick

- Na większości elementów można ustawić atrybut onclick zamiast addEventListener()
- Atrybut można ustawić tylko na jedną funkcję.
- Nie można podłączyć wielu obserwatorów.
- Uwaga na cudzysłowie i apostrofy!

```
<p onclick="alert('Clicked!')">Click here and see what happens!</p>  
<p id="other">Click here too!</p>  
<script>  
  let other = document.getElementById("other");  
  other.onclick = function () {  
    alert("Other clicked!");  
  }  
</script>
```

Usuwanie listenera

- Aby wyrejestrować obserwatora zdarzenia, należy znać jego nazwę.
 - Zamiast funkcji anonimowej, trzeba funkcję nazwać.
- Funkcja `removeEventListener` (nazwa, callback).

```
<p>Click here and see what happens!</p>
<button onclick="addListener()">Add listener</button>
<button onclick="removeListener()">Remove listener</button>

<script>
  let observed = document.querySelector("p");

  function listenerCallback() {
    alert("Hey!");
  }

  function addListener() {
    observed.addEventListener("click", listenerCallback);
  }

  function removeListener() {
    observed.removeEventListener("click", listenerCallback);
  }
</script>
```

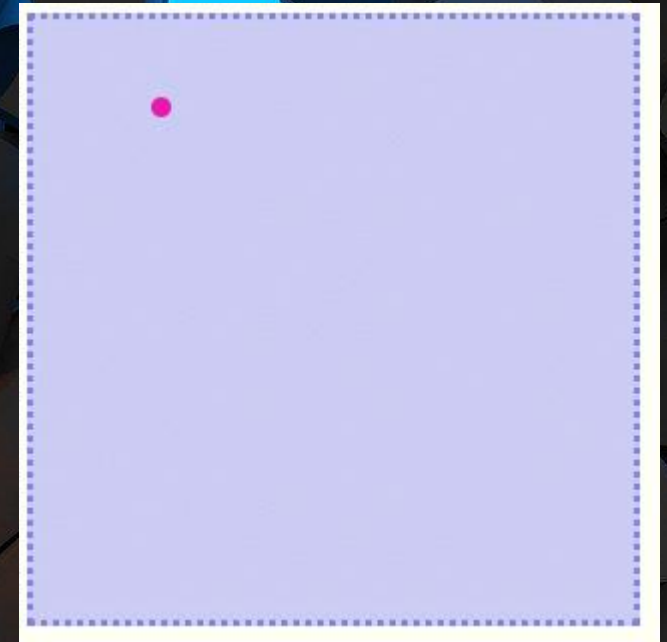
Obiekt event

- Funkcja callback może przyjmować parametr.
- Przy wykryciu zdarzenia, do funkcji przekazany parametr event.
- Obiekt zawiera dodatkowe informacje o zdarzeniu.
- Przykładowo:
 - pozycja X,Y gdzie kliknięto
 - czy wciśnięty Ctrl, Shift
 - jakie zdarzenie wykryto
 - type: „click”
- Właściwości zależne od typu zdarzenia.

```
<p>Click here and see what happens!</p>
<script>
  let observed = document.querySelector("p");
  observed.addEventListener("click", function(event) {
    event.preventDefault();
    console.log(event);
    //--> MouseEvent {screenX: 1668, screenY: -387,...}
  });
</script>
```

Wyzwanie

- Stwórz kontener i punkt wewnątrz niego.
- Po kliknięciu w kontenerze, punkt przechodzi do wskazanego miejsca.
- Zastosuj transition dla osiągnięcia animacji.



Propagacja zdarzeń

- Jeśli handler jest zarejestrowany na elemencie z dziećmi:
 - zdarzenie na dziecku
 - zostanie obsłużone przez handler zarejestrowany na rodzicu
- Jeśli i rodzic i dziecko ma zarejestrowany handler:
 - oba handlers zostaną uruchomione
 - w kolejności od miejsca zdarzenia (najbardziej szczegółowa lokalizacja) wzwyż
 - na koniec handlers zarejestrowane na obiekcie `window`
- `event.stopPropagation()` wstrzymuje propagację
- Większość zdarzeń myszy posiada właściwość `target`
 - Określa jaki element został kliknięty (`button`, `p`)
- Przykład

Blokowanie domyślnego zachowania przeglądarki

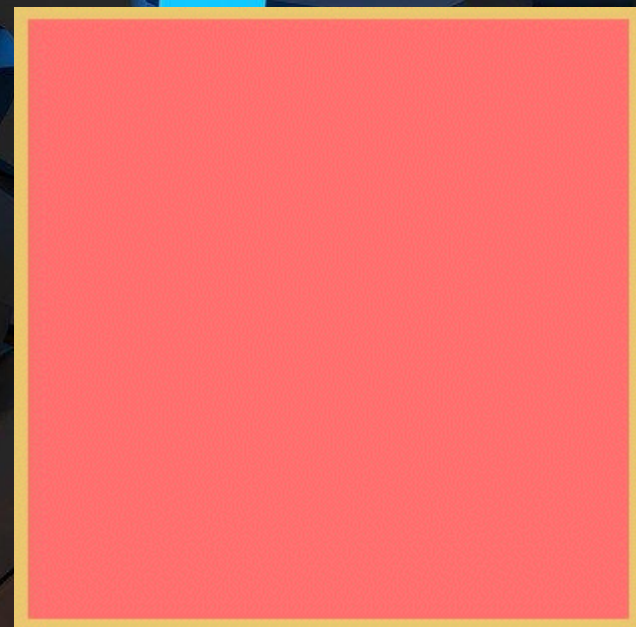
- Przeglądarka ma szereg domyślnych zachowań:
 - naciśnięcie strzałki w dół, w górę
 - przewija stronę
 - kliknięcie PPM
 - otwiera menu kontekstowe
- Handlery zdarzeń wykonywane przed zachowaniami domyślnymi.
- Możliwość stworzenia własnych:
 - skrótów klawiszowych
 - menu kontekstowego
 - blokowanie zachowań domyślnych:
 - `event.preventDefault()`
- Nie w każdej przeglądarce, nie wszystko da się nadpisać.

Zdarzenia na klawiszach

- `keydown`
 - gdy wciśnięty klawisz
 - powtarzany, dopóki klawisz nie zwolniony
- `keyup`
 - gdy klawisz zwolniony
- `keypress`
 - deprecated
- klawisz we właściwości `key`
 - specjalne klawisze mają swoje ciągi tekstowe („Enter”)
 - modyfikatory w `.shiftKey`, `.ctrlKey`, `.altKey`, `.metaKey`
- alternatywnie zdarzenia na `<input>` i `<textarea>`:
 - `input`

Wyzwanie

- Stwórz kontener.
- Zmieniaj kolor kontenera po wciśnięciu klawisza:
 - r – czerwony
 - g – zielony
 - b – niebieski



Zdarzenia myszy

- `mousedown`, `mouseup`
- `click`
 - po `mouseup`
 - wywoływany na najbliższym elemencie obejmującym oba `mousedown` i `mouseup`
 - przy przeciąganiu myszą – pozycja start i stop
- `dblclick`
 - podwójne kliknięcie
- właściwości:
 - `clientX`, `clientY`
 - pozycja w pikselach od górnego lewego narożnika okna
 - `pageX`, `pageY`
 - pozycja od narożnika dokumentu
 - inne od `clientX`, `clientY` jeśli dokument przewinięty
- `mousemove`
 - gdy mysz się porusza
 - umożliwia śledzenie pozycji myszy

Zdarzenia dotykowe

- `touchstart`
 - gdy palec rozpoczyna dotknięcie ekranu
- `touchmove`
 - przy przesuwaniu palca po ekranie
- `touchend`
 - przy oderwaniu palca od ekranu
- właściwość `touches`
 - obiekt z listą punktów dotyku
 - może być więcej niż 1
 - każdy ma swoje `clientX`, `clientY`, `pageX`, `pageY`
- przykład (widok responsywny)

Zdarzenia przewijania

- `scroll`
- Wykonuje się po fakcie przewinięcia ekranu.
- Przydatne zmienne globalne:
 - `innerHeight`
 - wysokość okna
 - `innerWidth`
 - szerokość dokumentu
 - `document.body.scrollHeight`
 - długość dokumentu
 - `pageYOffset`
 - aktualne przesunięcie (przewinięcie)

Wyzwanie

- Stwórz pasek postępu czytania artykułu.
- Wykorzystaj zdarzenie scroll.
- Przydatne zmienne globalne:
 - `innerHeight`
 - wysokość okna
 - `document.body.scrollHeight`
 - długość dokumentu
 - `pageYOffset`
 - aktualne przesunięcie (przewinięcie)

Inne zdarzenia

- `focus`
 - przy aktywacji elementu
- `blur`
 - przy opuszczeniu elementu
- `load`
 - gdy dokument wczyta się do końca
- `beforeunload`
 - przed samym opuszczeniem strony

XMLHttpRequest

XMLHttpRequest

- Interfejs umożliwiający JS wysyłanie żądań HTTP.
- Stworzony przez Microsoft w końcu lat 1990 dla Internet Explorer.
- Kolejne przeglądarki, by nie zostać w tyle, implementowały interfejs XMLHttpRequest.
- XMLHttpRequest stał się de facto standardem na wiele lat.

Żądania synchroniczne

- Utworzenie instancji obiektu XMLHttpRequest().
- Wybór metody, adresu URL, parametr
 - `async = false`.
- Wysłanie zapytania bez treści (`null`).
- Zwrócony przez serwer kod stanu HTTP i zawartość pobranego dokumentu dostępne we właściwościach instancji obiektu.
- Właściwości:
 - `req.status`
 - `req.statusText`
 - `req.responseText`
 - `req.responseXML`
 - `req.getResponseHeader(...)`

```
let req = new XMLHttpRequest();
req.open("GET", "./data.txt", false);
req.send(null);
// ...wait...

console.log(req.responseText);
//--> Topping candy jelly-o chocolate...
console.log(req.status);
//--> 200
console.log(req.statusText);
//--> OK
console.log(req.getResponseHeader('content-type'))
//--> text/plain
```

Zapytania asynchroniczne

- Parametr `async` funkcji `open` ustawiony na `true`.
- Konieczność subskrypcji zdarzenia `load` na instancji obiektu `XMLHttpRequest`.
- Funkcja `send()` zwraca kontrolę programu nie czekając na wczytanie danych.

```
let req = new XMLHttpRequest();
req.open("GET", "./data.txt", true);

req.addEventListener("load", function(event) {
  console.log(event);
  // (2) --> ProgressEvent {loaded: 151, total: 151, type: "load",...}

  // we use req!
  console.log(req.responseText);
  // (3) --> Topping candy jelly-o chocolate...
  console.log(req.status);
  // (4) --> 200
  console.log(req.statusText);
  // (5) --> OK
  console.log(req.getResponseHeader('content-type'))
  // (6) --> text/plain
});

req.send(null);
console.log("The show goes on!");
// (1) --> The show goes on!
```

Obiekt XML

```
<cookies>
  <cookie>candy</cookie>
  <cookie>jelly-o</cookie>
  <cookie>chocolate</cookie>
  <cookie>bonbon</cookie>
</cookies>
```

```
let req = new XMLHttpRequest();
req.open("GET", "./data.xml", true);

req.addEventListener("load", function(event) {
  let cookies = req.responseXML.querySelectorAll("cookie");
  for (let cookie of cookies) {
    console.log(cookie.childNodes[0].textContent);
  }
  //--> candy
  //--> jelly-o
  //--> chocolate
  //--> ...
});
req.send(null);
```

Parsowanie JSON

```
[  
  "candy",  
  "jelly-o",  
  "chocolate",  
  "bonbon"  
]
```

```
let req = new XMLHttpRequest();  
req.open("GET", "./data.json", true);  
  
req.addEventListener("load", function(event) {  
  let cookies = JSON.parse(req.responseText);  
  for (let cookie of cookies) {  
    console.log(cookie);  
  }  
  //--> candy  
  //--> jelly-o  
  //--> chocolate  
  //--> ...  
});  
req.send(null);
```

Fetch API

Promises

- Kod asynchroniczny bazuje na funkcjach callback.
- Funkcja wykonująca zapytanie asynchronicznie kończy się przed powrotem callbacka.
 - Kod robi się mało czytelny.
- Klasa standardowa `Promise`
 - obiekt reprezentujący przyszłe zdarzenie
 - wartość obietnicy może już być dostępna, a może dopiero się pojawić
 - upraszczają funkcje asynchroniczne
 - wygląd zwykłych funkcji – dane wejściowe i dane wyjściowe (których może jeszcze nie być)

Tworzenie Promise

- Konstruktor przyjmuje dwie funkcje:
 - resolve
 - reject
- then
 - obsługa sukcesu (gdy resolve)
- catch
 - obsługa błędów (gdy reject)
- Promise w jednym z 3 stanów:
 - pending
 - fulfilled
 - rejected
- then i catch zwracają kolejny Promise

```
const promise = new Promise( (resolve, reject) => {  
  setTimeout(() => {  
    resolve('After 2 seconds')  
  }, 2000);  
});
```

```
promise.then((value) => {  
  console.log(value);  
  // (2, two seconds later) --> After 2 seconds  
});
```

```
console.log(promise);  
// (1) --> [object Promise]
```

Reject

```
const promise = new Promise( (resolve, reject) => {  
  setTimeout(() => {  
    reject('Failed after 2 seconds')  
  }, 2000);  
});  
  
promise.then((value) => {  
  console.log(value);  
  // won't get there  
});  
  
promise.catch((reason) => {  
  console.log(reason);  
  //--> Failed after 2 seconds  
  
  return "returned from catch"  
});  
  
console.log(promise);  
//--> [object Promise]
```

Fetch API

- Proste API do wysyłania / pobierania danych asynchronicznie.
- Oparty na Promise'ach.
- Wywołanie:
 - `let promise = fetch(url, init)`
 - `init` – obiekt z parametrami
 - `method` – GET, POST, PUT, DELETE itp.
 - `headers` – obiekt z nagłówkami, przykładowo
 - `{„Content-Type”: „application/json”}`
 - `{„Content-Type”: „application/x-www-form-urlencoded”}`
 - `body` – dane, zgodnego typu z nagłówkiem Content-Type

GET – pobieranie tekstu

```
fetch('./data.txt')
  .then((response) => {
    console.log(response);
    //--> [object Response]
    console.log(response.body);
    //--> [object ReadableStream]

    return response.text();
  })
  .then((data) => {
    console.log(data);
    //--> Topping candy jelly-o ...
  })
;
```

GET – pobieranie JSON

```
[  
  "candy",  
  "jelly-o",  
  "chocolate",  
  "bonbon"  
]
```

```
fetch('./data.json')  
  .then((response) => {  
    console.log(response);  
    //--> [object Response]  
    console.log(response.body);  
    //--> [object ReadableStream]  
  
    return response.json();  
  })  
  .then((data) => {  
    console.log(data);  
    //--> ["candy", "jelly-o", "chocolate", "bonbon"]  
  })  
;
```

GET – pobieranie XML

```
<cookies>
  <cookie>candy</cookie>
  <cookie>jelly-o</cookie>
  <cookie>chocolate</cookie>
  <cookie>bonbon</cookie>
</cookies>
```

```
fetch('./data.xml')
  .then((response) => {
    return response.text();
  })
  .then((data) => {
    let parser = new DOMParser();
    let xmlDoc = parser.parseFromString(data, "text/xml");
    let cookies = xmlDoc.querySelectorAll("cookie");
    for (let cookie of cookies) {
      console.log(cookie.childNodes[0].textContent);
    }
    //--> candy
    //--> jelly-o
    //--> chocolate
    //--> ...
  })
;
```


Wyzwanie

- Pobierz i wyświetl dane z API Github.
 - Pobierz dane z <https://api.github.com/repos/ideaspotPL/ai1/commits>
 - Przeanalizuj strukturę zwracanych danych JSON.
 - Wybierz listę commitów.
 - Dla każdego commita, wyświetl pozycję listy:
 - autor: message commita

24.10.2025

42

Canvas

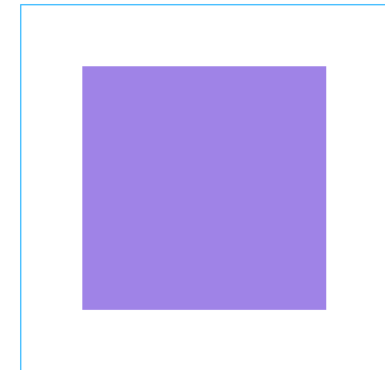
O canvas

- Zawarty w elemencie `<canvas>`
- Możliwość tworzenia grafiki rastrowej.
- Szerokość i wysokość określana w atrybutach `width` i `height`.
- Konieczność wskazania kontekstu:
 - `canvas.getContext()`
 - obsługiwane: „2d” i „webgl2”
- `fillStyle` – kolor wypełnienia
- `fillRect` – rysuj wypełniony prostokąt

```
<p>Above</p>
<canvas width="300" height="300"></canvas>
<p>Below</p>
```

```
<script>
  let canvas = document.querySelector("canvas");
  let context = canvas.getContext("2d");
  context.fillStyle = "#9f83e7";
  context.fillRect(50, 50, 200, 200);
</script>
```

Above



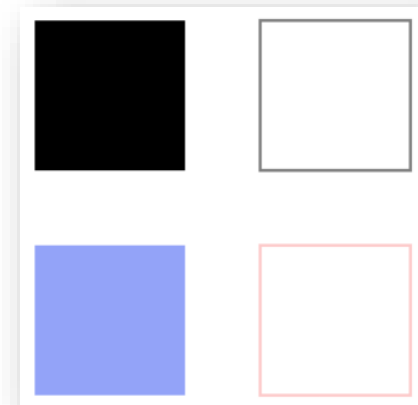
Below

Linie i powierzchnie

- W interfejsie canvas, kształt może być wypełniony (filled) lub tylko kreślony (stroked).
- `fillRect(x, y, w, h)`
 - wypełniony prostokąt
- `strokeRect(x, y, w, h)`
 - kreślony prostokąt
- Uwaga:
 - kolory, grubość linii itp. **nie są** przekazywane w funkcji
 - wykorzystywany stan kontekstu
- `fillStyle` – kolor wypełnienia
- `strokeStyle` – kolor linii
- `lineWidth` – grubość linii

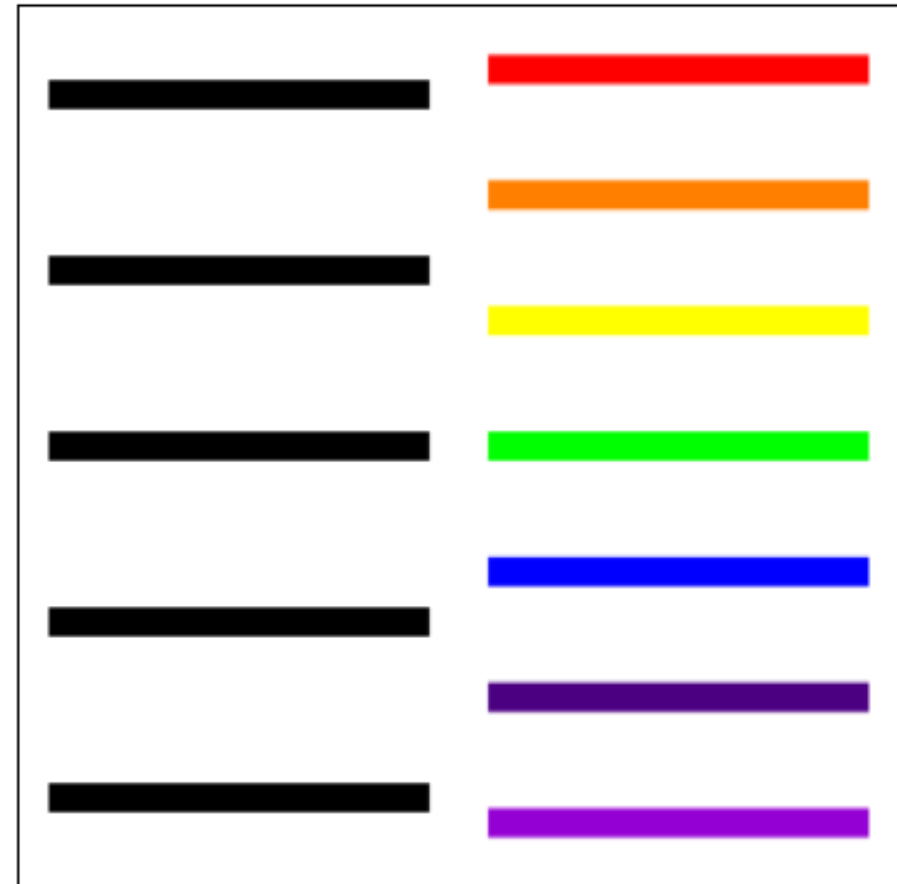
```
<canvas width="300" height="300"></canvas>
<script>
  let canvas = document.querySelector("canvas");
  let context = canvas.getContext("2d");
  context.fillRect(25, 25, 100, 100);
  context.strokeRect(175, 25, 100, 100);

  context.fillStyle = "#93a3f8";
  context.strokeStyle = "#fd9696";
  context.fillRect(25, 175, 100, 100);
  context.strokeRect(175, 175, 100, 100);
</script>
```



Ścieżki

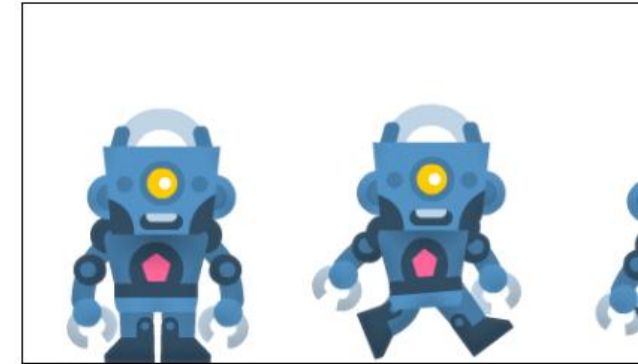
- Sekwencja linii.
- Rysowane za pomocą sekwencji wywołań metod:
 - `beginPath()`
 - rozpoczęcie ścieżki
 - `moveTo(x, y)`
 - określenie początku linii
 - `lineTo(x, y)`
 - określenie końca linii
 - `stroke()`
 - narysowanie wszystkich zaprogramowanych linii
- Wypełnianie ścieżek:
 - `fill()`
 - ścieżka się domknie
 - `closePath()` domyka ścieżkę jawnie



Obrázky

- <https://www.kenney.nl/assets/toon-characters-1>
- `context.drawImage()`
 - `img, dx, dy`
 - `img, dx, dy, dw, dh`
 - skalowanie
 - `img, sx, sy, sw, sh, dx, dy, dw, dh`
 - wycinanie i skalowanie
- `context.clearRect(x, y, w, h)`
- Przykład

Size 1:1



Scale 1:4



Single Image



Wyzwanie

- Zaimplementuj klasę wyświetlającą i animującą robota.
 - Robot <https://www.kenney.nl/assets/toon-characters-1>
 - Dwa stany przełączane spacją:
 - robot stoi
 - robot idzie
 - Notacja class



24.10.2025

48

WebGL

Jak powstał WebGL

- Vladimir Vukicevic
 - 2006 r.
 - narodowość Amerykańsko-Serbska
 - Rozpoczęcie prac nad implementacją OpenGL dla <canvas>
- Kronos Group
 - założone w efekcie, w marcu 2011
 - organizacja nonprofit
 - celem stworzenie WebGL
 - specyfikacja, by umożliwić przeglądarkom dostęp do GPU
 - Graphics Processing Units

WebGL2

- Do tworzenia standardu przystąpiły wszystkie przeglądarki.
- Oryginalnie WebGL oparty na OpenGL ES 2.0.
- Początkowo wsparcie w FF, Chrome, Opera, IE11.
 - Brak wsparcia w Safari i Safari Mobile ☹️
- Rozpoczęcie wsparcia WebGL w Safari w 2014 roku
- Obecnie używany WebGL 2 oparty na specyfikacji OpenGL ES 3.0
 - Dostępny w przeglądarkach od 2016 roku

Zalety WebGL

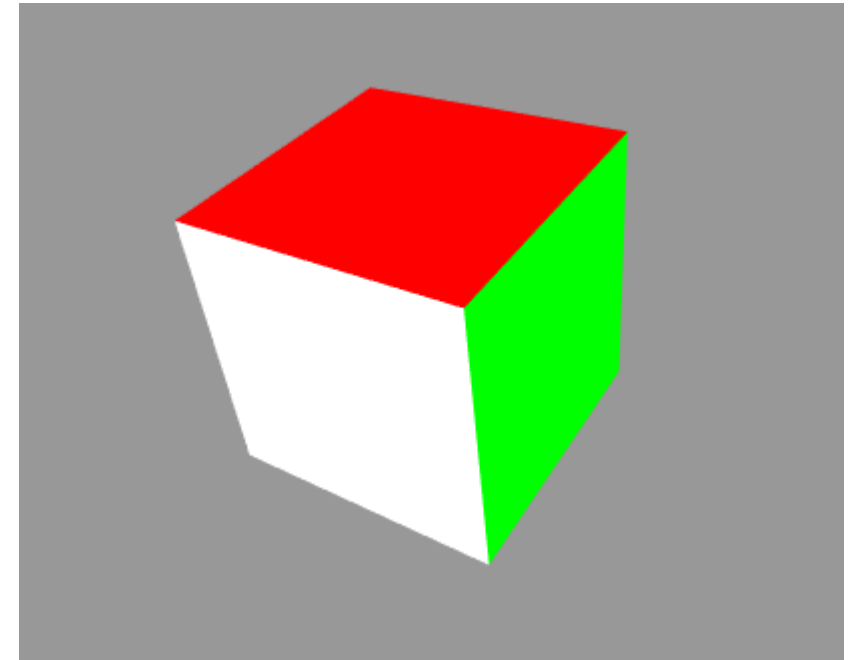
- Programowanie w JavaScript
 - język naturalny dla programistów
 - swobodny dostęp do drzewa DOM w przeglądarce
 - możliwość integracji WebGL z bibliotekami jak jQuery, React, Angular, Vue.js
- Automatyczne zarządzanie pamięcią
 - w przeciwieństwie do OpenGL, pamięcią i zakresem zarządza automatycznie JavaScript
 - łatwiejsze zrozumienie logiki aplikacji
- Wszechobecność
 - przeglądarki z obsługą JS instalowane domyślnie we wszystkich smartfonach i tabletach
 - możliwość publikowania aplikacji w szerokim ekosystemie urządzeń desktopowych i mobilnych
- Wydajność
 - wcześniej technologie renderowania 3D oparte na renderingu programowym
 - WebGL ma dostęp do lokalnej karty graficznej – renderowanie sprzętowe
- Brak kompilacji
 - WebGL pisany w JS – brak konieczności kompilacji
 - w shaderach konieczność kompilacji – ale wykonywana w karcie graficznej, a nie przeglądarce

Elementy w aplikacji WebGL

- canvas
 - miejsce w którym renderowana jest grafika
- obiekty
 - scena konstruowana jest z obiektów 3D
 - UWAGA: do WebGL przekazujemy listę trójkątów
- źródła światła
 - konieczne w świecie 3D żeby wyświetlać zawartość
 - obiekty 3D odbijają lub pochłaniają światło
- kamera
 - canvas służy za okno podglądu (viewport) przez które widzimy scenę
 - świat 3D widziany w perspektywie

Wszystko w trójkątach

```
var vertices = [  
  -1,-1,-1, 1,-1,-1, 1, 1,-1, -1, 1,-1,  
  -1,-1, 1, 1,-1, 1, 1, 1, -1, 1, 1,  
  -1,-1,-1, -1, 1,-1, -1, 1, 1, -1,-1, 1,  
  1,-1,-1, 1, 1,-1, 1, 1, 1, -1,-1, 1,  
  -1,-1,-1, -1,-1, 1, 1,-1, 1, 1,-1,-1,  
  -1, 1,-1, -1, 1, 1, 1, 1, 1, 1,-1,  
];  
  
var colors = [  
  5,3,7, 5,3,7, 5,3,7, 5,3,7,  
  1,1,3, 1,1,3, 1,1,3, 1,1,3,  
  0,0,1, 0,0,1, 0,0,1, 0,0,1,  
  1,0,0, 1,0,0, 1,0,0, 1,0,0,  
  1,1,0, 1,1,0, 1,1,0, 1,1,0,  
  0,1,0, 0,1,0, 0,1,0, 0,1,0  
];  
  
var indices = [  
  0,1,2, 0,2,3, 4,5,6, 4,6,7,  
  8,9,10, 8,10,11, 12,13,14, 12,14,15,  
  16,17,18, 16,18,19, 20,21,22, 20,22,23  
];
```



24.10.2025

SVG

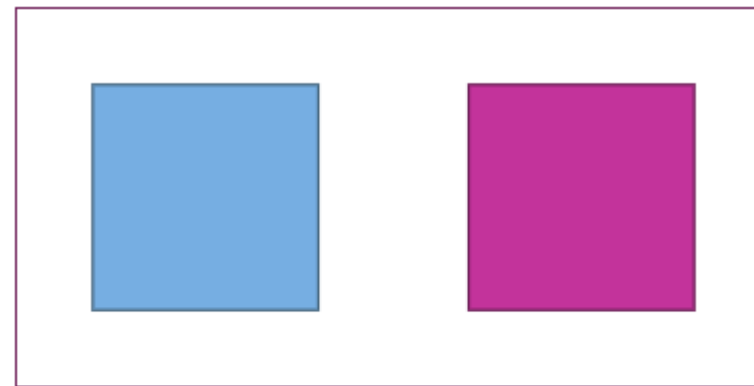
SVG

- Scalable Vector Graphics
- grafika skalowalna
 - wielka zaleta w świecie zróżnicowanych wielkości ekranów
- grafika wektorowa
 - w odróżnieniu od rastrowej
 - rysowane z wykorzystaniem operacji matematycznych
 - na ogół wydajniejsze i mniejsze od obrazków rastrowych
- format XML
 - kształty, linie i tekst opisane znacznikami
 - możliwość nawigacji po drzewie DOM

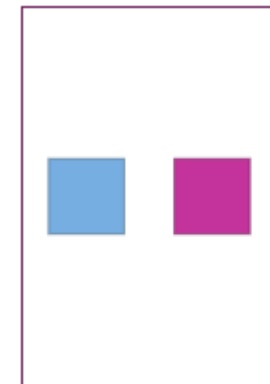
View Box

- Rozmiar obrazka na stronie ustawiany w atrybutach `width`, `height`.
- Rozmiar wyświetlanego płótna w obrazku w atrybucie `viewBox`.
 - Analogia ustawienia kamery na większym płótnie.

```
<svg x="0px" y="0px" width="300px" height="150px" viewBox="0 0 300 150">  
  <rect x="30" y="30" fill="#76AEE2" stroke="#44657E" width="90" height="90" />  
  <rect x="180" y="30" fill="#C3339B" stroke="#6D1D57" width="90" height="90" />  
</svg>
```



SVG Width 300
ViewBox Width 300

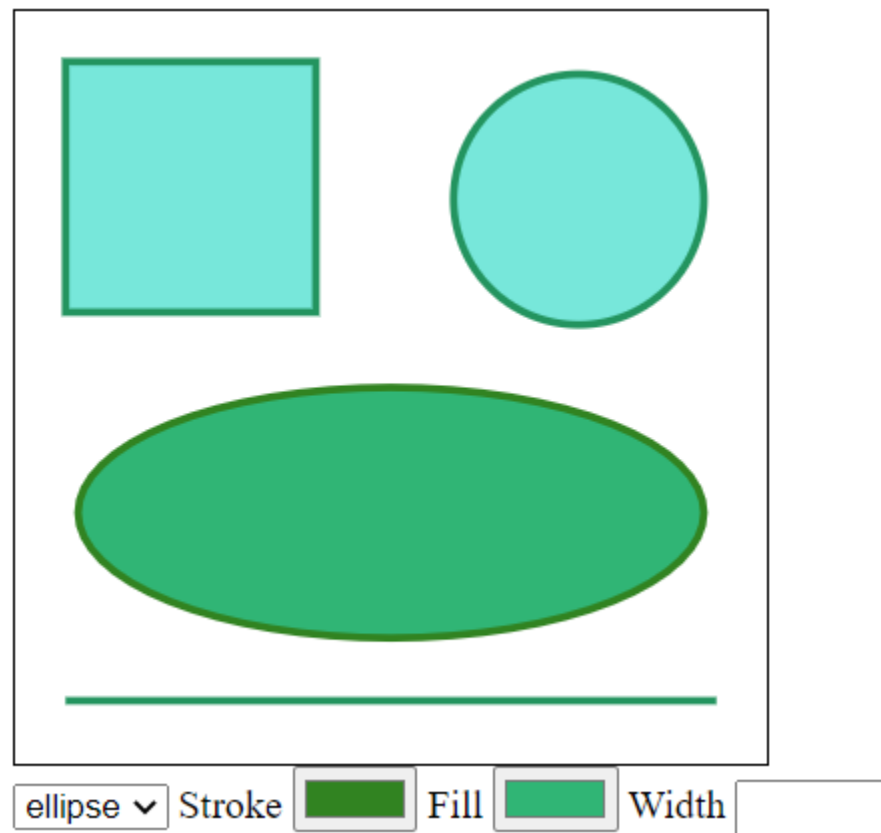


SVG Width 100
ViewBox Width 300

Kształty

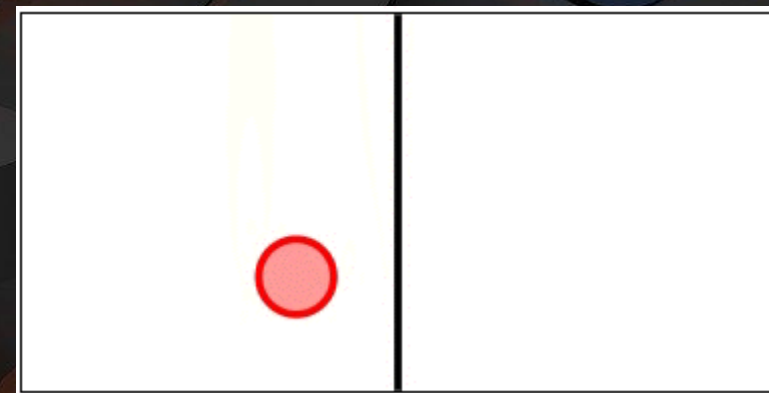
- `<rect>`
 - prostokąt
 - `x, y` – pozycja od górnego lewego rogu
 - `width, height`
 - `fill` – kolor wypełnienia
 - `stroke` – kolor linii
- `<circle>`
 - okrąg
 - `cx, cy` – środek okręgu
 - `r` – promień
 - `fill, stroke`
- `<g>`
 - grupa elementów
 - umożliwia nadawanie stylów dla wszystkich elementów grupy
- `<ellipse>`
 - elipsa
 - jak okrąg
 - `rx, ry` – dwa promienie
- `<polygon>`
 - wielokąt
 - `fill, stroke`
 - `points=„x1,y1 x2,y2 ...”` – punkty rozdzielane spacjami
- `<line>`
 - linia
 - `fill=„none”, stroke`
 - `x1, y1, x2, y2`

Kształty – przykład



Wyzwanie

- Animuj ruch piłki na obrazku <svg> z wykorzystaniem CSS.
- Przydatny CSS:
 - @keyframes
 - animation-...

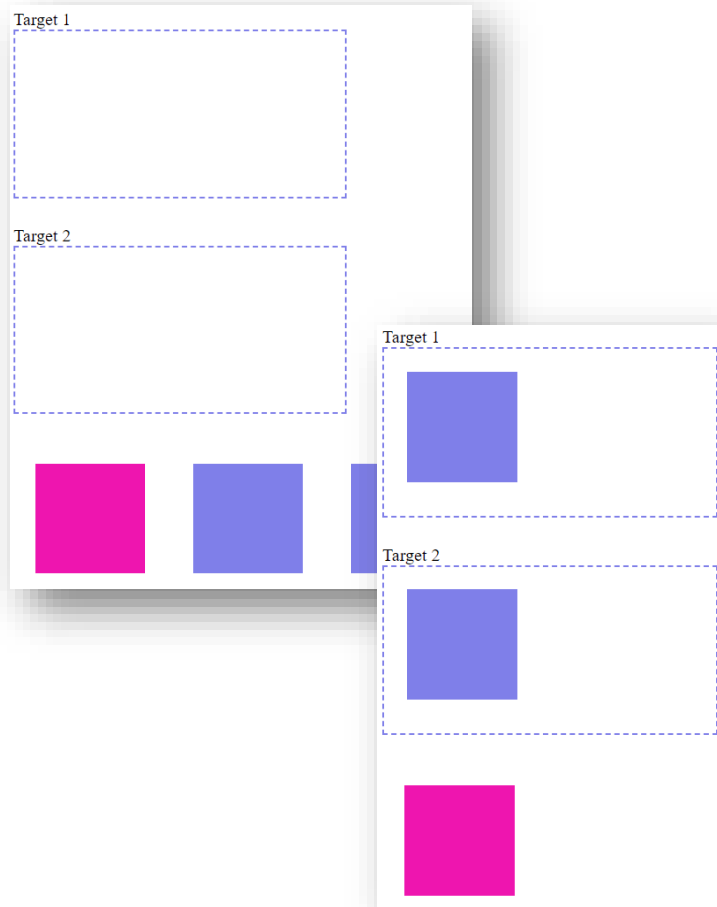


Drag-and-Drop API

Funkcjonalność drag-and-drop

- przeciągnij-i-upuść
- Atrybut `draggable=„true”`
 - włącza możliwość przeciągania elementów
 - obrazki i linki – przeciąganie domyślnie włączone
- Zdarzenia na elemencie przeciąganym:
 - `dragstart`
 - początek przeciągania
 - `dragend`
 - po wypuszczeniu elementu
- Zdarzenia na strefie zrzutu:
 - `dragenter`
 - gdy przeciągany element wejdzie nad strefę
 - `dragleave`
 - gdy przeciągany element opuści strefę
 - `dragover`
 - gdy przeciągany element ponad strefą
 - `drop`
 - gdy przeciągany element upuszczony do strefy

Przykład



Target 1
`<div class="drag-target"></div>`

Target 2
`<div class="drag-target"></div>`

```
<div id="non-draggable-item" class="item non-draggable" draggable="false"></div>
<div id="draggable-item-1" class="item draggable" draggable="true"></div>
<div id="draggable-item-2" class="item draggable" draggable="true"></div>
```

```
<script>
  let items = document.querySelectorAll('.item');
  for (let item of items) {
    item.addEventListener("dragstart", function(event) {
      this.style.border = "5px dashed #D8D8FF";
      event.dataTransfer.setData("text", this.id);
    });

    item.addEventListener("dragend", function(event) {
      this.style.borderWidth = "0";
    });
  }

  let targets = document.querySelectorAll(".drag-target");
  for (let target of targets) {
    target.addEventListener("dragenter", function (event) {
      this.style.border = "2px solid #7FE9D9";
    });
    target.addEventListener("dragleave", function (event) {
      this.style.border = "2px dashed #7f7fe9";
    });
    target.addEventListener("dragover", function (event) {
      event.preventDefault();
    });
    target.addEventListener("drop", function (event) {
      let myElement = document.querySelector("#" + event.dataTransfer.getData('text'));
      this.appendChild(myElement)
      this.style.border = "2px dashed #7f7fe9";
    }, false);
  }
</script>
```


Interfejsy obsługi audio i video



Obsługa multimedialnych

- HTML5 video:

- controls
- autoplay
- loop
- poster

- Kontrola źródła:

- video.src = ...
 - zmiana / wczytanie pliku
- zdarzenie loadeddata
 - wywoływane po wczytaniu pliku

- Odtwarzanie

- play()
- pause()
- currentTime
 - bieżący czas filmu w sekundach
- duration
 - łączny czas filmu w sekundach (float)
- volume (0 – 1)
 - głośność

- Zatrzymywanie filmu

- video.src = ""
- konieczność wyczyszczenia źródła

Wyzwanie

- Wykorzystaj API multimedialnych do obsługi odtwarzania pliku wideo.
- Do obsługi:
 - play / pause
 - stop
 - 5s do przodu / do tyłu
 - głośniej / ciszej



Przechowywanie danych

Local Storage i Cookies

LocalStorage

- Zbiór par nazwa-wartość.
- Wartości przechowywane w przeglądarce.
- Wartości zapisywane osobno dla każdej domeny.
- Wartości zapisywane w postaci plików.
 - Ograniczenie wielkości (5-10 MB).
 - Operacje I/O jednowątkowe.
- Obsługa:
 - `window.localStorage`.
 - `mojaZmienna = „MojaWartosc”;`
 - `[„mojaZmienna”] = „MojaWartosc”;`
 - `setItem(key, value)`
 - `getItem(key);`
 - `removeItem(key);`
 - `clear();`
 - działanie w bieżącej domenie
- Dane inne niż tekst:
 - `JSON.stringify(...)`
 - `JSON.parse(...)`

Ciasteczka

- Pozwalają na przechowywanie informacji na stronach.
- Zapisane w postaci plików tekstowych.
- Osobne dla każdej domeny.
- Ciasteczka przesyłane do serwera przy każdym zapytaniu.

name	value	add cookie
name	get cookie	
name	delete cookie	
get all cookies		

- Dostęp:
 - `let cookies = document.cookie`
 - lista ciasteczek rozdzielana ;
 - `document.cookie = „nazwa=wartość”;`
- Możliwość dodania:
 - czas wygaśnięcia:
 - `expires=Sun, 18 Oct 2020 12:00:00 UTC`
 - ścieżki
 - `path=`
 - ścieżka w domenie do której należy ciasteczko

24.10.2025

69

Geolokalizacja

Geolocation API

Geolokalizacja

- Jedno z najlepiej ustandaryzowanych API.
- Bazuje w znacznym stopniu na Google Gears API z 2008 r.
- Informacje pobierane poprzez zapytanie HTTP w tle, do Location Information Server.
- Lokalizacja otrzymywana na podstawie:
 - publicznego adresu IP
 - hotspotów WiFi
 - identyfikatorów MAC urządzeń Bluetooth
 - GPS
 - identyfikatorów anten GSM/CDMA

Dostęp

- Weryfikacja, czy przeglądarka obsługuje geolokalizację:
 - `if (navigator.geolocation) {}`
- Pobieranie bieżącej pozycji
 - `navigator.geolocation.getCurrentPosition(success, error);`
 - jednokrotne pobranie w danym momencie
 - `navigator.geolocation.watchPosition(success, error, options);`
 - `PositionOptions`:
 - `enableHighAccuracy: false, timeout: 5000, maximumAge: 0`
 - `navigator.geolocation.clearWatch(id)`
- Zwracana wartość:
 - `pos.`
 - `coords.`
 - `latitude`
 - `longitude`

Przykład

```
<button id="getLocation" onclick="getLocation()">Get Location</button>
<div id="positionInfo">
  <strong>Latitude:</strong> <span id="latitude"></span><br>
  <strong>Longitude:</strong> <span id="longitude"></span><br>
</div>

<script>
  function getLocation() {
    if (! navigator.geolocation) {
      alert("Sorry, no geolocation available for you!");
    }

    navigator.geolocation.getCurrentPosition((position) => {
      document.getElementById("latitude").innerText = position.coords.latitude;
      document.getElementById("longitude").innerText = position.coords.longitude;
    }, (positionError) => {
      console.error(positionError);
    }, {
      enableHighAccuracy: false
    });
  }
</script>
```

Get Location

Latitude: 53.4468676666666684

Longitude: 14.553462583333333

24.10.2025

73

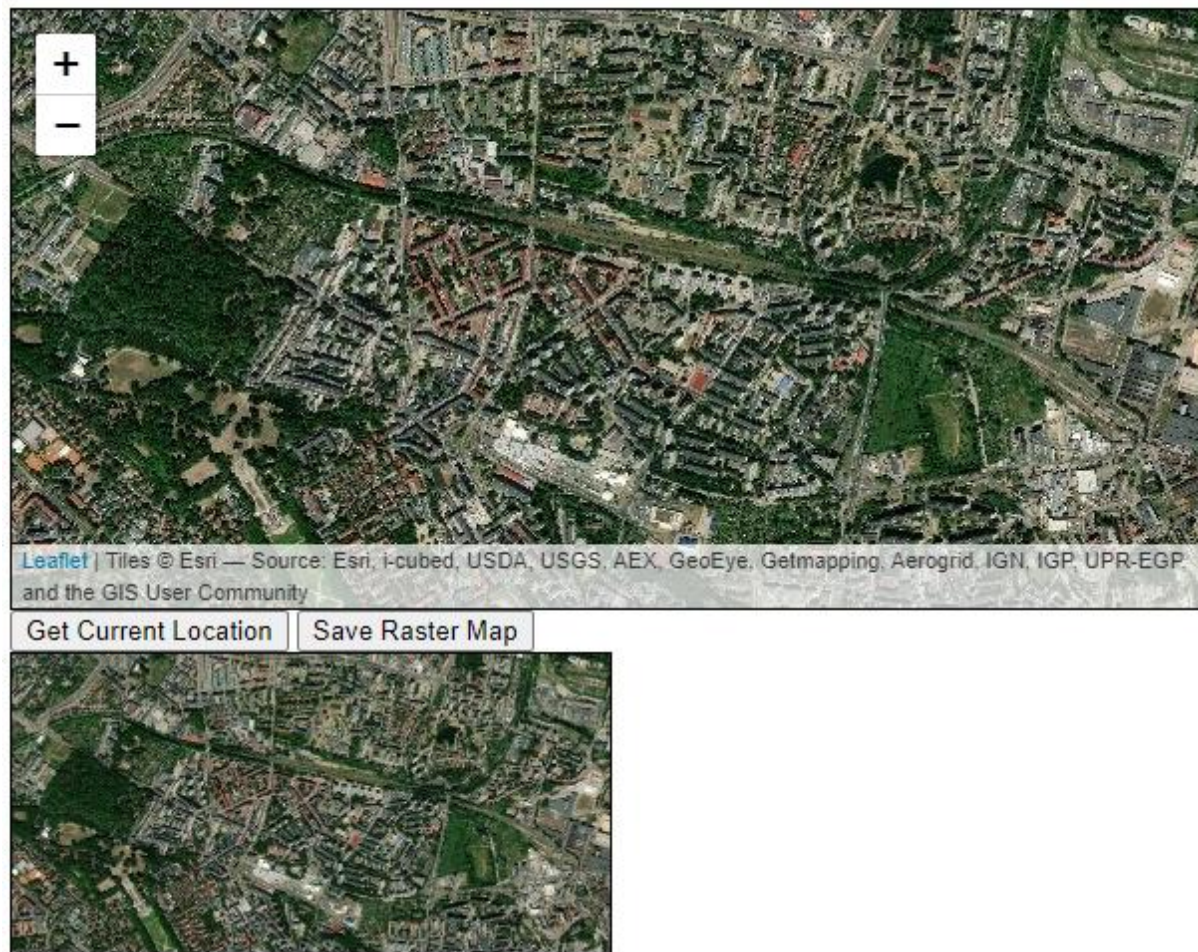
Mapy

Leaflet

Leaflet

- Wiodąca biblioteka JS do obsługi interaktywnych map na urządzenia mobilne i stacjonarne.
- Open-source.
- Rozmiar około 39 KB
- Zaprojektowana z myślą o:
 - wygodzie użytkowania
 - wydajności
 - użyteczności.
- Możliwość rozszerzania z wykorzystaniem pluginów.

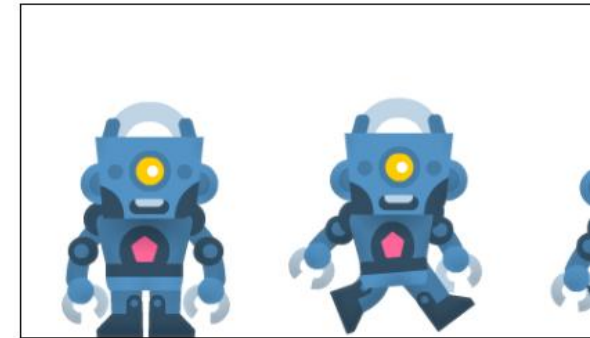
Przykład



Podsumowanie

- Zdarzenia i ich obsługa
- Połączenia asynchroniczne: XMLHttpRequest i Fetch API
- Grafika rastrowa w Canvas
- Grafika wektorowa w SVG
- Animacje SVG
- Drag-and-Drop API
- Obsługa multimedialnych
- Geolokalizacja i mapy

Size 1:1



Scale 1:4



Single Image



24.10.2025

77

Pytania?

Ostatnia szansa 😊

Post-TEST

W jaki sposób w JS można rozpocząć nasłuchiwanie zdarzenia „click” na elemencie:

- A. `element.addEventListener(„click”, (event) => {...})`
 - można podpiąć dokładnie 1 listener
- B. `element.addEventListener(„click”, (event) => {...})`
 - można podpiąć różne listenery wiele razy
- C. atrybut `onclick=...` elementu
 - można podpiąć wiele listenerów w kolejnych atrybutach `onclick`



<https://ispot.link/ai1po5>

Spis ilustracji

- <https://pixabay.com/images/id-2492009/> (wyzwanie)